

1.3. The CUDA platform

The NVIDIA CUDA platform consists of many pieces of software and hardware and many important technologies developed to enable computing on heterogeneous systems. This chapter **serves to introduce some of the fundamental concepts and components of the CUDA platform that are important for application developers** to understand. This chapter, like [Programming Model](#), is not specific to any programming language, but applies to everything that uses the CUDA platform.

1.3.1. Compute Capability and Streaming Multiprocessor Versions

Every NVIDIA GPU has a **Compute Capability (CC) number**, which **indicates what features are supported** by that GPU and specifies **some hardware parameters for that GPU**. These specifications are documented in the [Section 5.1](#) appendix. A list of all NVIDIA GPUs and their compute capabilities is maintained on the [CUDA GPU Compute Capability page](#).

Compute capability is denoted as a major and minor version number in the format X.Y where X is the major version number and Y is the minor version number. For example, CC 12.0 has a major version of 12 and a minor version of 0. The compute capability directly corresponds to the version number of the SM. For example, the SMs within a GPU of CC 12.0 have SM version *sm_120*. This version is used to label binaries.

[Section 5.1.1](#) shows how to query and determine the compute capability of the GPU(s) in a system.

1.3.2. CUDA Toolkit and NVIDIA Driver

The **NVIDIA Driver** can be thought of as the operating system of the GPU. The NVIDIA **Driver** is a software component which must be **installed on the host system's operating system** and is **necessary for all GPU uses**, including display and graphical functionality. The NVIDIA Driver is foundational to the CUDA platform. In addition to CUDA, the NVIDIA Driver provides all other methods of using the GPU, for example Vulkan and Direct3D. The NVIDIA Driver has version numbers such as r580.

The **CUDA Toolkit** is a set of libraries, headers, and tools for **writing, building, and analyzing software** which utilizes GPU computing. The CUDA Toolkit is **a separate software product from the NVIDIA driver**

The **CUDA runtime** is a special case of **one of the libraries provided by the CUDA Toolkit**. The CUDA runtime provides both an API and some language extensions to handle common tasks such as allocating memory, copying data between GPUs and other GPUs or CPUs, and launching kernels. The API components of the CUDA runtime are referred to as the CUDA runtime API.

The [CUDA Compatibility](#) document provides full details of compatibility between different GPUs, NVIDIA Drivers, and CUDA Toolkit versions.

1.3.2.1. CUDA Runtime API and CUDA Driver API

The CUDA runtime API is implemented on top of a lower-level API called the *CUDA driver API*, which is an API exposed by the NVIDIA Driver. **This guide focuses on the APIs exposed by the CUDA runtime API**. All the same functionality can be achieved using only the driver API if desired. Some features are only available using the driver API. Applications may use either API or both interoperably. Section [The CUDA Driver API](#) covers interoperation between the runtime and driver APIs.

The full API reference for the CUDA runtime API functions can be found in the [CUDA Runtime API Documentation](#) .

The full API reference for the CUDA driver API can be found in the [CUDA Driver API Documentation](#) .

1.3.3. Parallel Thread Execution (PTX)

A fundamental but sometimes invisible layer of the CUDA platform is the **Parallel Thread Execution (PTX) virtual instruction set architecture (ISA)**. PTX is **a high-level assembly language for NVIDIA GPUs**. PTX provides an abstraction layer over the physical ISA of real GPU hardware. Like other platforms, applications can be written directly in this assembly language, though doing so can add unnecessary complexity and difficulty to software development.

Domain-specific languages and compilers for high-level languages can generate PTX code as an intermediate representation (IR) and **then use NVIDIA's offline or just-in-time (JIT) compilation tools to produce executable binary GPU code**. This enables the CUDA platform to be programmable from languages other than just those supported by NVIDIA-provided tools such as [NVCC: The NVIDIA CUDA Compiler](#).

Since GPU capabilities change and grow over time, the PTX virtual ISA specification is versioned. PTX versions, like SM versions, correspond to a compute capability. For example, PTX which supports all the features of compute capability 8.0 is called *compute_80*.

Full documentation on PTX can be found in the [PTX ISA](#) .

1.3.4. Cubins and Fatbins

CUDA applications and libraries are usually written in a higher-level language like C++. That higher-level language is compiled to PTX, and then the PTX is compiled into **real binary for a physical GPU, called a CUDA binary, or cubin for short**. A cubin has a specific binary format for a specific SM version, such as *sm_120*.

Table of Contents

CUDA Programming Guide	^
1. Introduction to CUDA	^
1.1. Introduction	
1.2. Programming Model	
1.3. The CUDA platform	
2. Programming GPUs in CUDA	^
3. Advanced CUDA	^
4. CUDA Features	^
5. Technical Appendices	^
6. Notices	

On this page

1.3.1. Compute Capability and Streaming Multiprocessor Versions
1.3.2. CUDA Toolkit and NVIDIA Driver
1.3.3. Parallel Thread Execution (PTX)
1.3.4. Cubins and Fatbins

[Back to top](#)

Executables and library binaries that use GPU computing contain both CPU and GPU code. The GPU code is stored within a container called a *fatbin*. Fatbins can contain cubins and PTX for multiple different targets. For example, an application could be built with binaries for multiple different GPU architectures, that is, different SM versions. When an application is run, its GPU code is loaded onto a specific GPU and the best binary for that GPU from the fatbin is used.

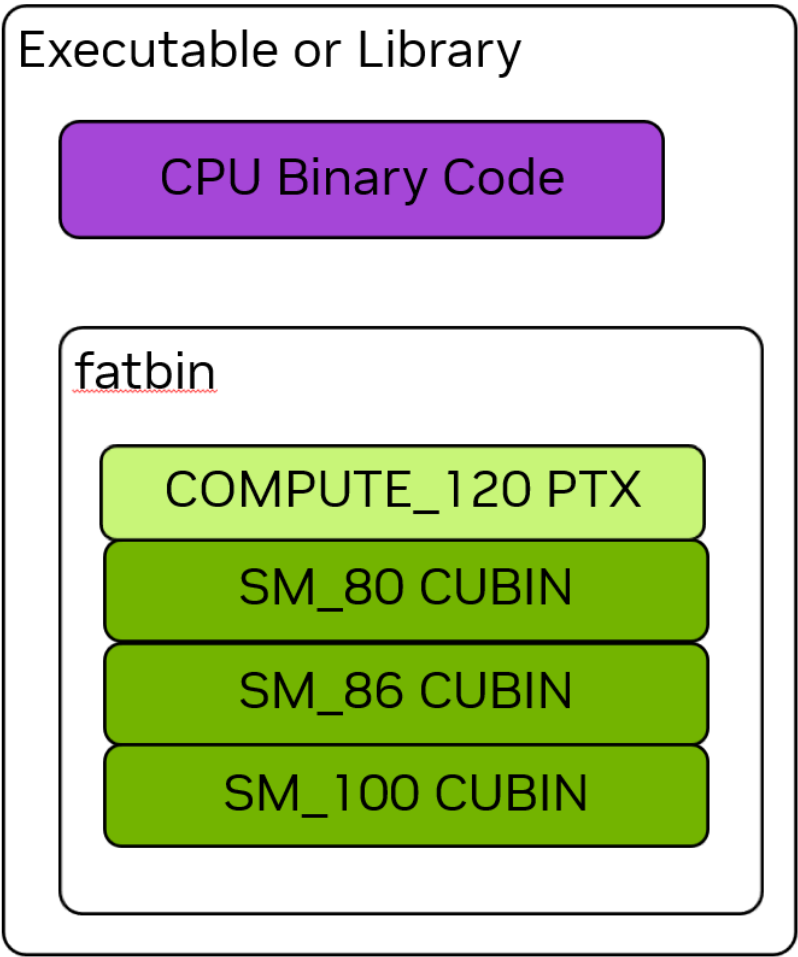


Figure 8: The binary for an executable or library contains both CPU binary code and a fatbin container for GPU code. A fatbin can contain both cubin GPU binary code and PTX virtual ISA code. PTX code can be JIT compiled for future targets.

Fatbins can also contain one or more PTX versions of GPU code, the use for which is described in PTX Compatibility. Figure 8 shows an example of an application or library binary which contains multiple cubin versions of GPU code as well as one version of PTX code.

1.3.4.1. Binary Compatibility

NVIDIA GPUs guarantee binary compatibility in certain circumstances. Specifically, within a major version of compute capability, GPUs with minor compute capability greater than or equal to the targeted version of cubin can load and execute that cubin. For example, if an application contains a cubin with code compiled for compute capability 8.6, that cubin can be loaded and executed on GPUs with compute capability 8.6 or 8.9. It cannot, however, be loaded on GPUs with compute capability 8.0, because the GPU's CC minor version, 0, is lower than the code's minor version, 6.

NVIDIA GPUs are not binary compatible between major compute capability versions. That is, cubin code compiled for compute capability 8.6 will not load on GPUs of compute capability 9.0.

When discussing binary code, the binary code is often referred to as having a version such as *sm_86* in the above example. This is the same as saying the binary was built for compute capability 8.6. This shorthand is often used because it is how a developer specifies this binary build target to the NVIDIA CUDA compiler, *nvcc*.

Note

Binary compatibility is promised only for binaries created by NVIDIA tools such as `nvcc`. Manual editing or generating binary code for NVIDIA GPUs is not supported. Compatibility promises are invalidated if binaries are modified in any way.

1.3.4.2. PTX Compatibility

GPU code can be stored in executables in binary or PTX form, which is covered in Cubins and Fatbins.

When an application stores the PTX version of GPU code, that PTX can be JIT compiled at application runtime for any compute capability equal or higher to the compute capability of the PTX code. For example, if an application contains PTX for *compute_80*, that PTX code can be JIT compiled to later SM versions, such as *sm_120* at application runtime. This enables forward compatibility with future GPUs without the need to rebuild applications or libraries.

1.3.4.3. Just-in-Time Compilation

PTX code loaded by an application at runtime is compiled to binary code by the device driver. This is called just-in-time (JIT) compilation. Just-in-time compilation increases application load time, but allows the application to benefit from any new compiler improvements coming with each new device driver. It also enables applications to run on devices that did not exist at the time the application was compiled.

When the device driver just-in-time compiles PTX code for an application, it automatically caches a copy of the generated binary code in order to avoid repeating the compilation in subsequent invocations of the application. The cache - called the compute cache - is automatically invalidated when the device driver is upgraded, so that applications can benefit from the improvements in the new just-in-time compiler built into the device driver.

How and when PTX is JIT compiled at runtime has been relaxed since the earliest versions of CUDA, allowing more flexibility for when and if to JIT compile some or all kernels. The section Lazy Loading describes the available options and how to control JIT behavior. There are also a few environment variables which control just-in-time compilation behavior, as described in CUDA Environment Variables.

